basis of the expression of the ‘when’ block and returns results for each record. Here I will use the search CASE syntax to implement the pivot table.

Stored virtual tables

SQL supports virtual tables. It contains no data but generates reports when executed. Tables are created using select queries and stored as SQL query files. SQL never stores data and remains inactive till it gets executed. In that sense, it is useful to store complex queries and is often used to generate tables, joining multiple tables based on specific relationships among their fields.

The SQL CREATE or REPLACE VIEW syntax is as follows:

```
CREATE VIEW view_name AS
SELECT column1, column2,
FROM table_name
WHERE condition;
CREATE OR REPLACE VIEW view_name AS
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

Views can be deleted as follows:

```
DROP VIEW  view_name;
```

Stored virtual tables are often used in high level query and report generation. Here I shall use the invoice virtual table for pivot table generation.

Resource

In this discussion I have used the Northwind database for pivot table implementation. Since it is well known to the database user community and the organisation of the tables is easy to understand, I hope a little browsing of the table will help readers to understand the implementation of a pivot table. Readers can download the database from *https://code.google.com/archive/p/northwindextended/downloads*. In this discussion I have used the *Northwind.MySQL5* database.

Pivot table

Though a pivot table may have different forms, at its very core, it is a transposition of data from multiple rows into columns of a single row. This approach to rearrangement is particularly common in data analysis for report generation. To illustrate the technique in SQL over a table, here I have considered the orders table of the Northwind database. This table contains order-wise detailed information about different items for several years. If we want a report of month-wise total freight charges of every year, we can make ‘year’ as the pivot to transform the table orders into a per-year month-wise freight pivot table. To extract month-wise total freight charges, you need to run the query over a sub-query of orders, with the year and month extracted from the *orderDate* field. On the basis of extracted month values, we can now take the total of the month-wise freight charges and populate them in the 12 columns of the report. The following SQL example will make this clear.

Simple pivot table

Let’s have a look at the table orders. It has the field ‘freight’ indicating cargo charges for the shipment of orders. To have month-wise total freight charges for each year, we need a pivot table. To have a summary report on the months, we need a sub-query on *OrderDate* to extract the year and month so that the pivot table can be created for month-wise total freight charges.

```
CREATE TABLE `orders` (
`OrderID` INT(11) NOT NULL AUTO_INCREMENT,
`CustomerID` VARCHAR(5) NULL DEFAULT NULL,
`EmployeeID` INT(11) NULL DEFAULT NULL,
`OrderDate` DATETIME NULL DEFAULT NULL,
`RequiredDate` DATETIME NULL DEFAULT NULL,
`ShippedDate` DATETIME NULL DEFAULT NULL,
`ShipVia` INT(11) NULL DEFAULT NULL,
`Freight` DECIMAL(10,4) NULL DEFAULT ‘0.0000’,
`ShipName` VARCHAR(40) NULL DEFAULT NULL,
`ShipAddress` VARCHAR(60) NULL DEFAULT NULL,
`ShipCity` VARCHAR(15) NULL DEFAULT NULL,
`ShipRegion` VARCHAR(15) NULL DEFAULT NULL,
`ShipPostalCode` VARCHAR(10) NULL DEFAULT NULL,
`ShipCountry` VARCHAR(15) NULL DEFAULT NULL,
PRIMARY KEY (`OrderID`),
INDEX `OrderDate` (`OrderDate`),
INDEX `ShippedDate` (`ShippedDate`),
INDEX `ShipPostalCode` (`ShipPostalCode`),
INDEX `FK_Orders_Customers` (`CustomerID`),
INDEX `FK_Orders_Employees` (`EmployeeID`),
INDEX `FK_Orders_Shippers` (`ShipVia`),
CONSTRAINT `FK_Orders_Customers` FOREIGN KEY (`CustomerID`)
REFERENCES `customers` (`CustomerID`),
CONSTRAINT `FK_Orders_Employees` FOREIGN KEY (`EmployeeID`)
REFERENCES `employees` (`EmployeeID`),
CONSTRAINT `FK_Orders_Shippers` FOREIGN KEY (`ShipVia`)
REFERENCES `shippers` (`ShipperID`)
)
ENGINE=InnoDB
```

The CASE statements of the main query will get months from the sub-query and will perform the aggregation of freight charges for the months of the year. This is shown in